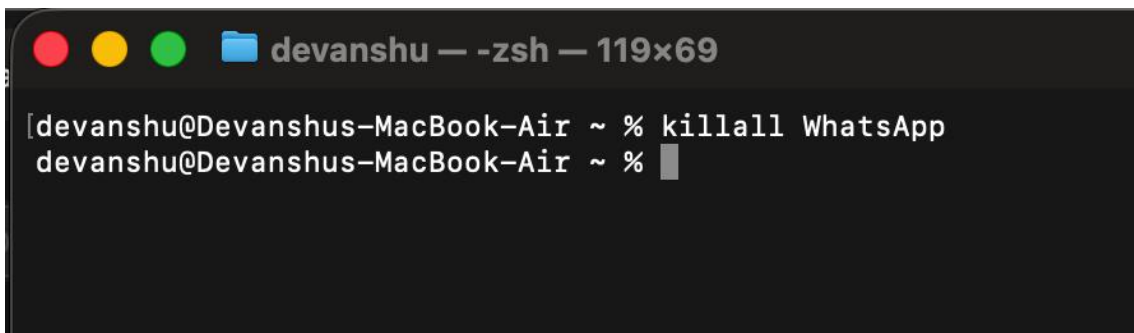


OS Practical No. 4

Aim :

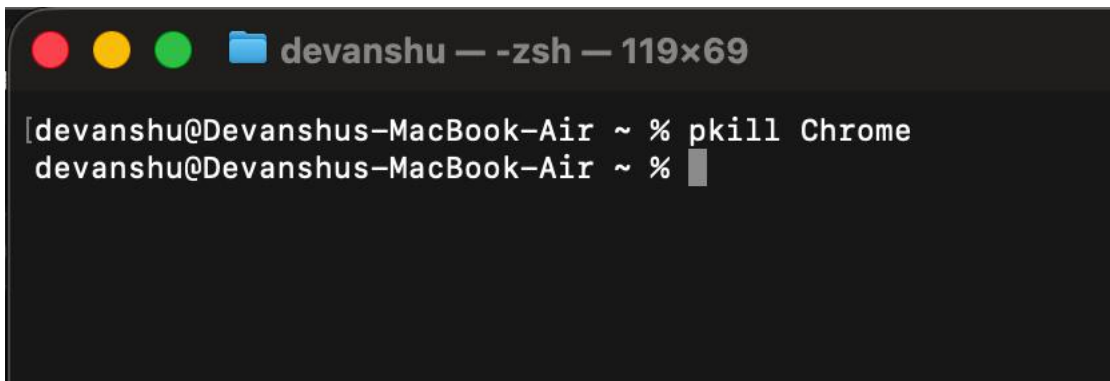
1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

Kill processes by name



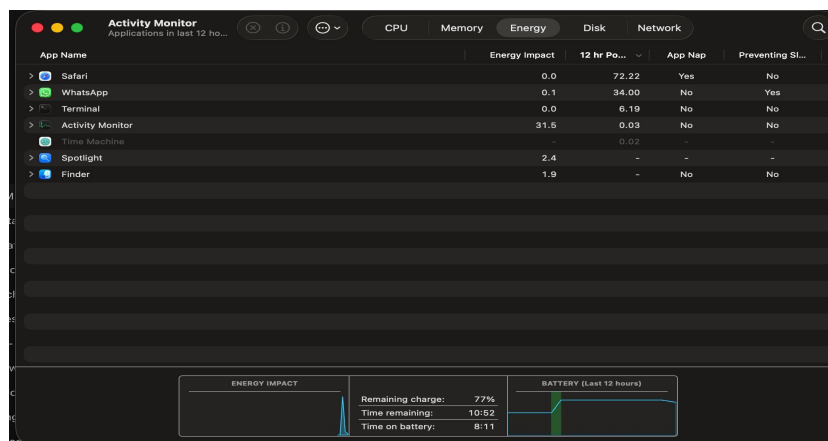
```
devanshu — -zsh — 119x69  
[devanshu@Devanshus-MacBook-Air ~ % killall WhatsApp  
devanshu@Devanshus-MacBook-Air ~ %
```

Kill a process based on the process name



```
devanshu — -zsh — 119x69  
[devanshu@Devanshus-MacBook-Air ~ % pkill Chrome  
devanshu@Devanshus-MacBook-Air ~ %
```

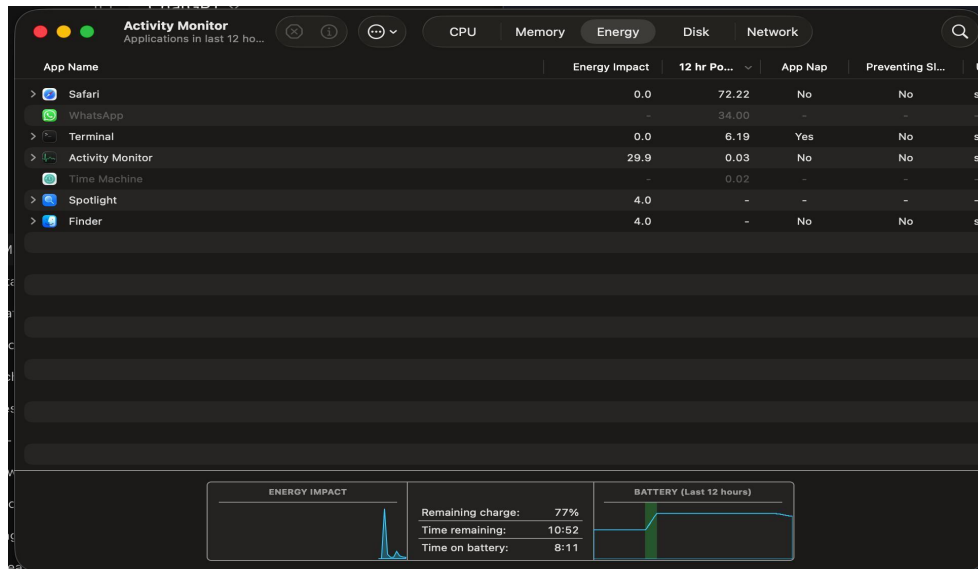
Kill a single process at a time with the given process ID



```
devanshu — -zsh — 119x69

devanshu@Devanshus-MacBook-Air ~ % kill -9 PID

kill: illegal pid: PID
devanshu@Devanshus-MacBook-Air ~ %
```



2. Write a program for process creation using C
- Orphan Process

Code :

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
    pid_t pid = fork();
    if (pid > 0) {
        printf("Parent process exiting\n");
    }
    else if (pid == 0)
    { sleep(5);
        printf("Child process (Orphan) running\n");
    }
    return 0;
}
```

```
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
devanshu@Devanshus-MacBook-Air ~ % nano hello.c

devanshu@Devanshus-MacBook-Air ~ % gcc hello.c -o hello_program

devanshu@Devanshus-MacBook-Air ~ % ./hello_program

Parent process exiting
devanshu@Devanshus-MacBook-Air ~ % Child process (Orphan) running

devanshu@Devanshus-MacBook-Air ~ % █
```

- Zombie Process

Code :

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child process exiting\n");
    }
    else {
        sleep(10);
        printf("Parent process running\n");
    }
    return 0;
}
```

```
devanshu — -zsh — 119x69

[devanshu@Devanshus-MacBook-Air ~ % nano Zombie.c
[devanshu@Devanshus-MacBook-Air ~ % gcc Zombie.c -o Zombie_program
[devanshu@Devanshus-MacBook-Air ~ % ./Zombie_program
Child process exiting

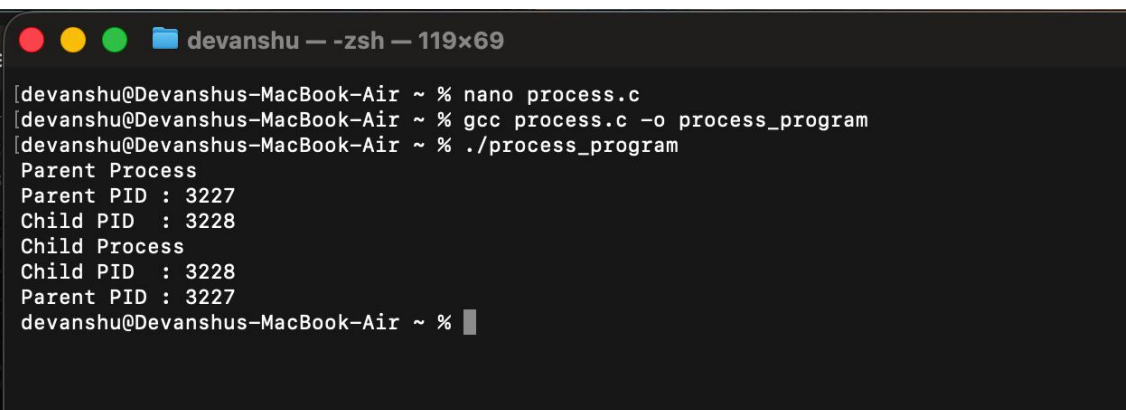
^[[C^[[DParent process running
devanshu@Devanshus-MacBook-Air ~ %
devanshu@Devanshus-MacBook-Air ~ % █
```

3. Create the process using fork () system call.

- Child Process creation
- Parent process creation
- PPID and PID

Code :

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
    }
    else if (pid > 0) {
        // Parent process
        printf("Parent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
    }
    else {
        printf("Fork failed\n");
    }
    return 0;
}
```



The screenshot shows a terminal window titled "devanshu — zsh — 119x69". The user has executed the following commands:

```
[devanshu@Devanshus-MacBook-Air ~ % nano process.c
[devanshu@Devanshus-MacBook-Air ~ % gcc process.c -o process_program
[devanshu@Devanshus-MacBook-Air ~ % ./process_program
```

The output of the program is as follows:

```
Parent Process
Parent PID : 3227
Child PID : 3228
Child Process
Child PID : 3228
Parent PID : 3227
devanshu@Devanshus-MacBook-Air ~ %
```